Progress Report: Sprint 11

CS.028 Optimizing Research Software Codes - 30 March 2026
Kathryn Butler, Joseph Schaab, Michael McAllister

Summary

- Continued UI improvements by redirecting warnings away from the console and improving the accuracy of timestamps.
- Continued calculate_positions() optimizations by restructuring the data concatenation and replacing the bottleneck apply() panda pattern.
- Began drafting our project poster.

Sprint Work Log

UI Improvements

Status: Ongoing (85% complete)
Author: Kathryn Butler
Reviewers: N/A
Source Link: Commit 56
Supporting Artifact:

```
t value for compat will change from compat='no_conflicts' to compat='override'. This is likely to lead to different results when combinin
verlapping variables with the same name. To opt in to new defaults and get rid of these warnings now use `set_options(use_new_combine_kwa
defaults=True) or set compat explicitly.
  obs = xarray.merge((obs, o))
C:\Projects\optimizing-research-software-codes\gnss_python-main\georinex\obs2.py:54: FutureWarning: In a future version of xarray the def
t value for compat will change from compat='no_conflicts' to compat='override'. This is likely to lead to different results when combinin
verlapping variables with the same name. To opt in to new defaults and get rid of these warnings now use `set_options(use_new_combine_kwa
defaults=True) or set compat explicitly.
  obs = xarray.merge((obs, o))
Processing started...
Using 8 cores to calculate positions.
If you encounter memory errors, reduce num_cores and try it again or disable parallel
C:\Users\kated\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-package
umpy\_core\fromnumeric.py:57: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'Data
me.transpose' instead.
  return bound(*args, **kwds)
```

Original Output

```
10:03:39 - ====================================
10:03:39 - GNSS RINEX Processing Started
10:03:39 - ====================================
10:03:39 - Loading config...
10:03:44 - Large obs version 3 file detected!
10:03:44 - Converting obs version down to 2...
```

```
10:05:31 - opening 178.12244 MByte SEPT2620.25O
10:05:42 - Processing RINEX files...
10:05:50 - Calculating positions using 8 cores...
Calculating satellite positions: 100%| 32/32 [11:44<00:00, 22.02s/it]
Building output rows: 100%| 1749600/1749600 [04:37<00:00, 6298.83it/s]
10:22:35 - Processing completed.
10:22:35 - Total runtime: 1136.80 seconds (19 minutes).
10:22:35 - ================================
```

Improved Output

Notes: The original output is extremely convoluted and confusing for users to understand. I've continued work on testing how increased partitions work for performance versus increased visualization. I am also optimizing the performance by removing the conversion to RINEX2DB, but so far any progress that has been made seems to make the program hang indefinitely.

Improved calculate_positions() Performance Logic

Status: Ongoing (60% complete)
Author: Joseph Schaab
Reviewers: N/A
Source Link: calculate_postions() - Documentation
Supporting Artifact:

```python
def calculate_positions(self):
    try:
      if self.config['etc'].get('disable_parallel', False):
        raise Exception('Throw exception to disable parallel')
      num_procs = self.config['etc'].get('num_cores', 0)

      if num_procs == 0:
        num_procs = multiprocessing.cpu_count() // 2
      logging.info(f'Calculating positions using {num_procs} cores...')
      chunks = num_procs * 4
      obs_partition = np.array_split(self.obs, chunks)

      with multiprocessing.Pool(processes=num_procs) as pool:
        results = list(
            tqdm(
                pool.imap(self.get_satellite_position_process_worker, obs_partition),
                total=len(obs_partition),
                desc="Calculating satellite positions",
            )
        )

      self.obs = pd.concat(results)

    except:
      import traceback
      traceback.print_exc()
      print('Unable to process parallel. Use single core instead')
      self.obs[['PosX', 'PosY', 'PosZ', 'ClkCorr']] = self.obs.apply(lambda x: self.get_satellite_position(x), axis=1,
                                                                     result_type='expand')
```

Original calculate_positions()

```python
def calculate_positions(self):
    try:
        if self.config['etc'].get('disable_parallel', False):
            raise Exception('Parallel disabled by config')

        num_procs = self.config['etc'].get('num_cores', 0) or (multiprocessing.cpu_count() // 2)
        logging.info(f'Calculating positions using {num_procs} cores...')

        obs_partition = np.array_split(self.obs, num_procs)

        from joblib import Parallel, delayed
        results = Parallel(n_jobs=num_procs)(
            delayed(self.get_satellite_position_process_worker)(chunk)
            for chunk in obs_partition
        )

        self.obs = pd.concat(results, copy=False)

    except Exception:
        import traceback
        traceback.print_exc()
        print('Unable to process in parallel. Falling back to single core.')

        rows = [self.get_satellite_position(row) for row in self.obs.itertuples()]
        self.obs[['PosX', 'PosY', 'PosZ', 'ClkCorr']] = rows
```

Updated calculate_positions()

Notes: Reworked chunking with pool.imap to use chunksize instead. This allows the preprocessing to be more organized and cut down some overhead runtime. Found through testing that pd.concat after parallelism is highly expensive. Reduced this runtime cost by preallocating and filling a subset of columns. Most importantly, I

researched the apply() panda pattern and discovered it to be a very slow process. I improved this by using intertuples() which is about 5-10 times faster. One issue is that I still need to rework some of the new improvements to coexist properly in the program's environment.

Risk and Quality Tracking

Bug Count: 11 (Link to issue query)

Top Bugs:

- Treating Keys as Positions and Using Argsort
  - **Severity:** High
  - **Owner:** Kathryn Butler
  - **Current Status:** Resolved
  - **Actions:** No further action needed. (Series.getitem treating keys as positions is deprecated. Look up how to solve it.)
- Incorrect conversion from float to datetime
  - **Severity:** Medium
  - **Owner:** Kathryn Butler
  - **Current Status:** Resolved
  - **Actions:** No further action needed. (Research into the line causing the problem. Determine the input causing the problem, and correct the datetime variable.)
- Log.txt hardcoded
  - **Severity:** Medium
  - **Owner:** Kathryn Butler
  - **Current Status:** Resolved
  - **Actions:** No further action needed. (Log.txt seems to be announcing a preset number of cores to calculate. Figure out why it is set to only be 8 cores. Find out if you can manually adjust the core count to be something other than just 8, then add the text to the function dynamically.)
- Inconsistent timing data
  - **Severity:** High
  - **Owner:** Kathryn Butler
  - **Current Status:** Ongoing
  - **Actions:** Create a smaller test data pool, run the tests multiple times to determine if the size of the file increases the likelihood of failed parsing attempts.
- Xarray.merge conflict
  - **Severity:** Medium
  - **Owner:** Kathryn Butler

    - **Current Status:** Ongoing
    - **Actions:** Set compat explicitly to something other than ‘override’, then test output for clarity. (Previous goal: Research what “compat” does, and either set it explicitly or use the new combine_kwarg_defaults.)

Next Sprint Plan

- Finalize IO. This includes making the program state that it has begun running immediately after startup (not 15 seconds later), and finishing the progress bars.
- Continue working on the bug list. Work on removing the warnings, and look into updating the libraries associated with the functions.
- Integrate the calculate_positions() updates into the program. Push the updates to the github repository.

Team Process Reflection

The current blockers we are facing are that the optimizations for calculate_positions() are difficult to integrate into the code. Additionally, due to the extremely limited team size, our capabilities to create a fully-optimized codebase has been hindered quite a bit.

To compensate for these shortcomings, we have communicated with our project partner and have focused on polishing the overall IO and updating the documentation to pass to future developers. We are also aiming to work on this project until the end of May to continue optimizations until the final deadline, instead of stopping at Sprint 13.